HireRoom: Build Status, Fixes and Resume Parsing

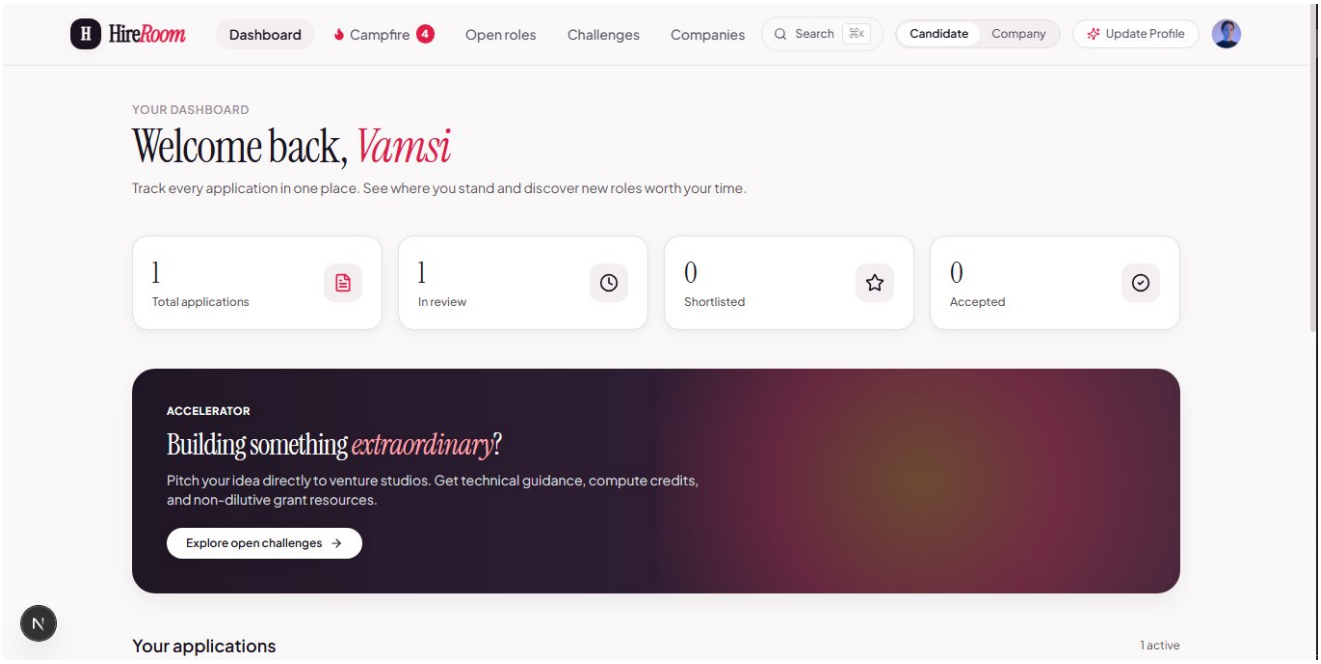
Where the Antigravity build stands (about 30 to 40 percent), what is wrong, how resume parsing really works, the page-access rules, and the fix order. Prepared 5 Nov 2026.

What this is based on. Four screenshots of the local build (localhost:3000) and what you told me: another resume gives the same profile data, and pages meant for before sign-up are reachable after it. I cannot see the code, so every item below is an observation from a screenshot or a symptom you reported. The agent must confirm each one in the code first (prompt R0 does that).

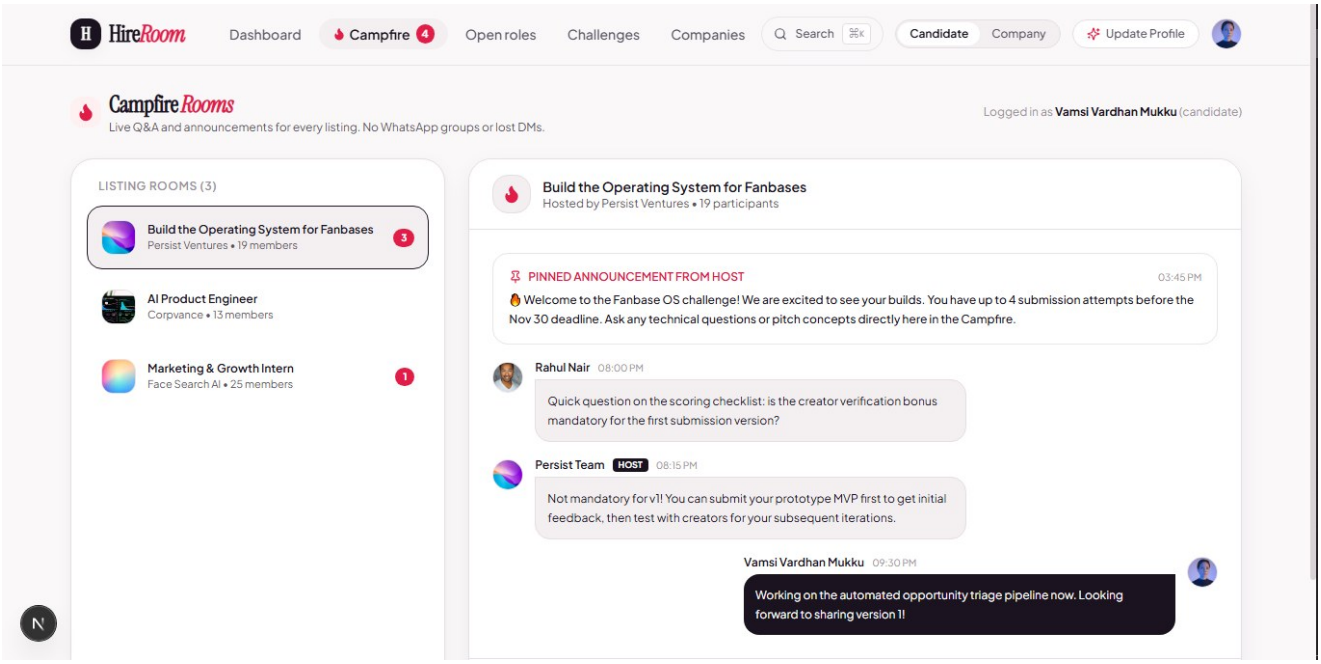
1. Where the build is now

Area	What looks right	What is wrong or unknown
Brand and layout	Logo, crimson accent, serif headings with an italic accent word, pill nav, rounded cards. This follows the direction.	Content width differs: the dashboard column is narrower than Roles and Campfire. The full app nav shows during onboarding.
Candidate dashboard	Four stat cards and an applications list.	The banner copy is a close paraphrase of Startupathon's, and it promises venture studios, compute credits and grants that HireRoom does not offer.
Open roles	Search, Jobs / Internships / Open Challenges filters, Remote only, cards with type badges.	The three cards are Startupathon's listings and companies (Persist Ventures, Corpvance, Face Search AI) with **Verified** badges. That shows real companies as verified on your product.
Campfire	Rooms list, pinned host announcement, HOST badge, own message bubble.	The conversation looks seeded (names, times, participant counts). Confirm messages are stored, arrive live, and counts are real. "Logged in as ... (candidate)" looks like a debug line.
Candidate onboarding	Editable profile with skill chips and a location field.	You reported the same data for a different resume, so parsing is likely mocked or cached (section 2). The phone field shows +91 twice and a leading 0.
Role switch	Candidate / Company toggle in the nav.	Any user can flip it. Role should come from the account and be enforced on the server.
Not in the screenshots	Sign-up, login, company posting, challenge page, submissions, emails, admin.	Unknown. Treat as unbuilt until the audit proves otherwise.

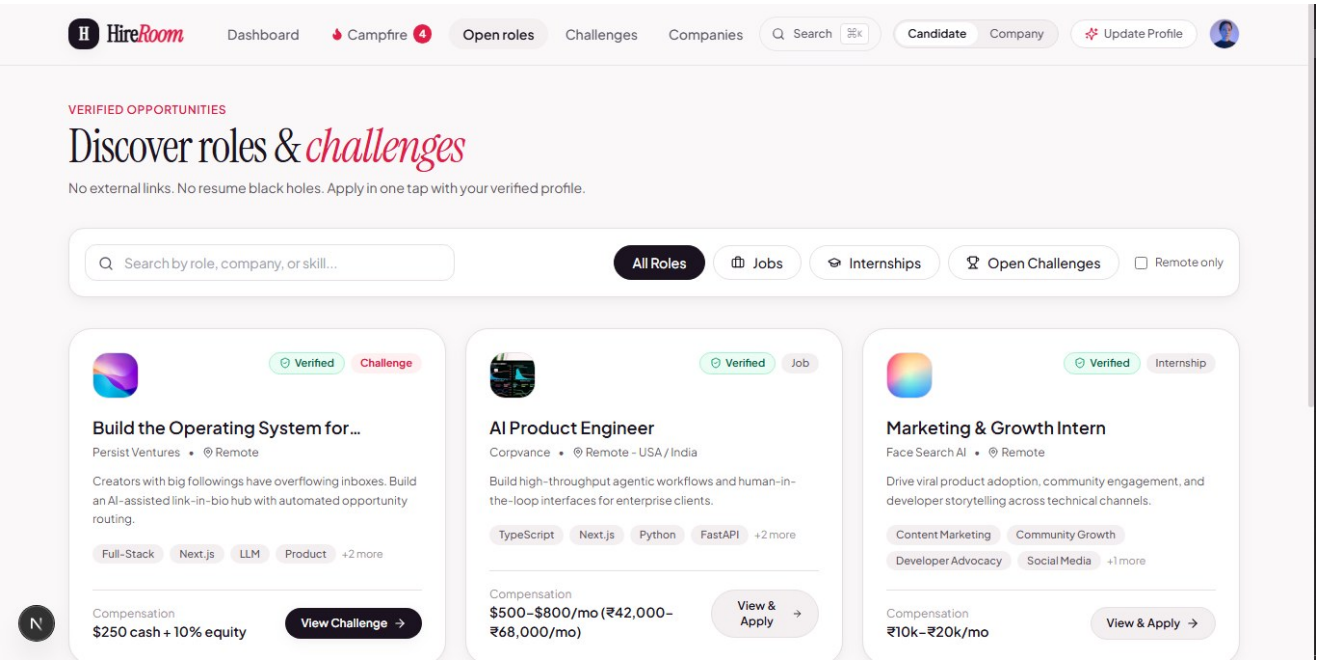
Screens reviewed



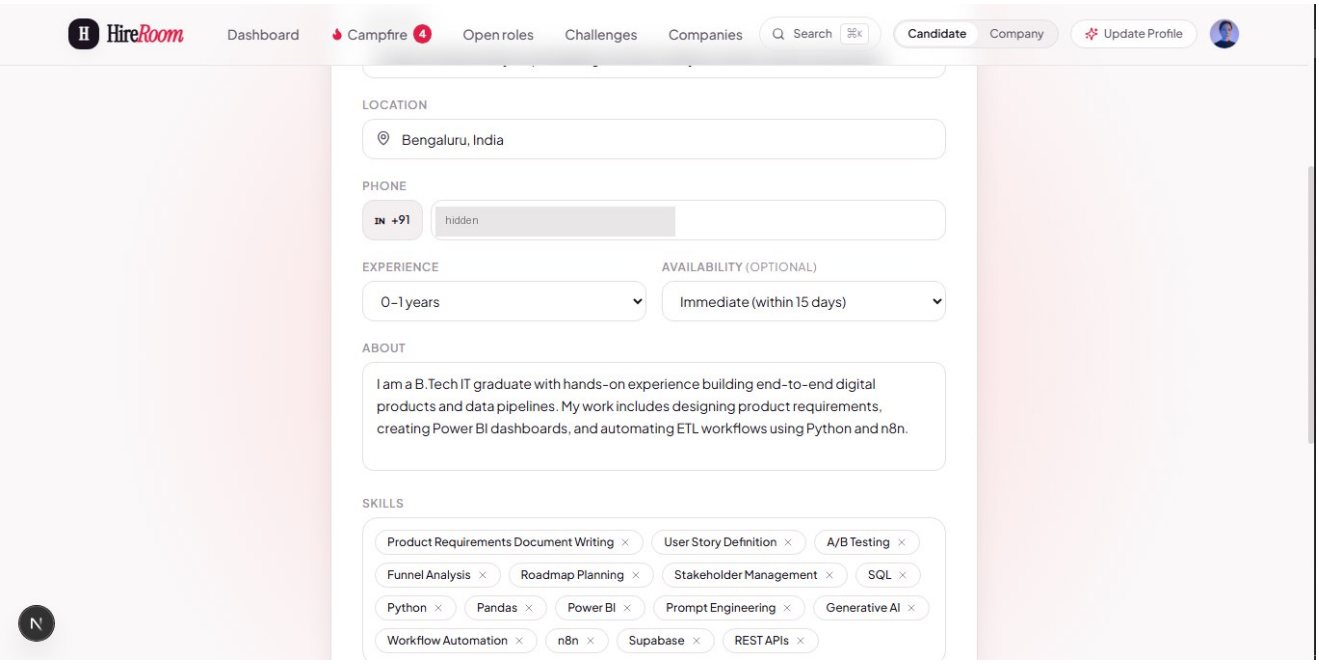
Candidate dashboard. Note the banner copy and the full nav.



Campfire. Seeded-looking conversation and a debug-style "Logged in as" line.



Open roles. Real companies' listings with Verified badges, and a wider page column than the dashboard.



Onboarding step 5 with the full nav still visible. The phone value is hidden in this copy; it showed the country code twice and a leading 0.

2. How resume parsing works, and why yours repeats

The standard pipeline

These stages come from vendor explainers, so treat their accuracy claims as unverified. The stages themselves are standard.

Stage	What happens	Where it goes wrong
1. Ingestion	File becomes text. PDFs store characters with positions, not in reading order. DOCX keeps paragraph order. Scanned PDFs need OCR.	Two-column layouts get interleaved, odd fonts garble text, scans misread characters.
2. Sections	Split into contact, experience, education, skills using header words, font cues and position.	Headers like "What I've Built" are missed by word lists.

Stage	What happens	Where it goes wrong
3. Entities	Email and phone by regex (most reliable). Dates by patterns. Titles and companies by named-entity models. Skills by a taxonomy list (tens of thousands of terms) plus context.	"Go" or "R" misread; negations ("no Kubernetes") still match.
4. Normalization	Map variants to one name: JS to JavaScript, Postgres to PostgreSQL.	Out-of-date taxonomies miss new tools.
5. Confidence	Score each field so shaky ones are flagged for the user.	Older parsers skip this and store a garbled title as if it were certain.

Three generations exist: rule-based (fast, brittle), NLP models (better, needs training data) and LLM-based (handles odd layouts and infers skills from context, but costs more and can invent a skill). One vendor explainer estimates that about 15% of PDFs need special handling, which is why showing the result for the user to check matters. Source: Reqcore explainer, Equip explainer.

What Unstop and similar platforms do

I could not find a public description of Unstop's parser, so I will not claim one. Platforms generally either buy a commercial parsing API (RChilli and Affinda are examples) or run a pipeline like the one above. For HireRoom, the LLM route is the practical choice: one Claude call with a JSON schema replaces most of the rule-based work.

Why your upload returns the same data

Check these in order. The audit prompt (R0) traces them in the code.

1. The parse function returns a fixture or hard-coded sample when a key is missing or the call fails (a silent fallback).
2. The result is stored per user and not overwritten (insert only when empty), or the form loads the existing profile first and ignores the new parse.
3. The uploaded file is never actually read (wrong path or name, or an old file is read).
4. The form is pre-filled from a saved draft or local storage.

Test that proves it is fixed: upload an engineer's resume, a designer's resume and a plain-text file. The three profiles must differ.

Recommended pipeline for HireRoom

Step	Tool	Notes
1. Validate and store	Supabase Storage	PDF or DOCX, up to 10 MB, keep a file hash.
2. Extract text	unpdf or pdfjs-dist for PDF, mammoth for DOCX	Check each license in the lockfile before adding.
3. Scanned files	Docling (MIT) as a small Python service, or send page images to Claude	If the extracted text is very short, treat the file as scanned.
4. Structure	Claude API with a JSON schema	"Only what is written. Null if missing. Never invent skills."
5. Validate	Zod, plus regex for email and phone	Reject impossible dates.
6. Normalize	Your own alias map (JS to JavaScript)	Cap at 25 skills. Grow the map over time.
7. Flag	Confidence per field	Highlight low-confidence fields as "check this".
8. Save	Overwrite parsed fields after confirmation	"Replace your profile with this resume?" when the profile already has data.
9. Log	parse_runs table	File hash, model, status, time. No sample-data fallback.

Open-source options

Project	License	Use it for
Docling (69.8k stars)	MIT	PDF, DOCX and image input with OCR, output as Markdown or JSON. Python, so run it as a separate service. Best for scanned or messy files.
Resume-Matcher (28.9k stars)	Apache-2.0	A reference for ATS checks (it runs PDFs through pdfminer) and prompts that forbid invented skills. Its page does not describe how it parses into fields.
OpenResume (8.9k stars)	AGPL-3.0	Has a TypeScript parser. Do not copy its code: AGPL can force you to open-source your hosted product.
pyresparser	Not verified	An older spaCy-based parser. I could not confirm its license or maintenance, so skip it.

3. Page access: before and after sign-up

This is the table the agent must enforce and test. Redirect, do not just hide.

Area	Logged out	Signed in, onboarding not done	Signed in and onboarded
Landing, /login, /signup	Allowed	Redirect to onboarding	Redirect to dashboard
Browse roles, challenges, companies	Allowed, read-only. Apply asks to sign in and returns to the same page	Redirect to onboarding	Allowed
Onboarding	Redirect to login	Allowed, only the user's own role flow	Redirect to dashboard. Edits go through Update Profile
Dashboard, Campfire, applications, profile	Redirect to login	Redirect to onboarding	Allowed
Company area (/company/*)	Redirect to login	Redirect to onboarding	Company role only
Admin area (/admin/*)	Redirect to login	Redirect to onboarding	Emails in ADMIN_EMAILS only

- Enforce in server middleware (called proxy in newer Next.js; check the installed version's docs), again in server components and API routes, and with row-level security. Hiding links in the client is not security.
- Use **getUser()** on the server, which verifies the token, rather than trusting the session cookie alone.
- Role is chosen at sign-up and stored on the account. Remove the nav toggle, or allow it only for an account that really has both a candidate profile and a company membership.
- Hide the main nav during onboarding and keep one consistent page width everywhere.
- Write one Playwright test per cell of the table.

4. Remaining work, in order, with the repo for each job

#	Task	Use	Done when
R0	Audit, no new features	Read the code, write docs/AUDIT.md	Every page and route is marked real or mocked.
R1	Access rules and roles	@supabase/ssr, Next middleware, Playwright	Every cell in the section 3 table passes.
R2	Real resume parsing	Claude API, unpdf or pdfjs-dist, mammoth, Docling for scans	Three different resumes give three different profiles.
R3	Clean data and forms	TanStack Form + Zod, libphonenumber-js	No Startupathon-derived names or copy. Phone saved as +91XXXXXXXXXX.
R4	Auth and emails	Supabase Send Email hook, React Email + Resend	The 15 events in the setup guide send and are logged.
R5	Campfire for real	Supabase Realtime Chat block + messages table	Two browsers see each other live and history survives a reload.
R6	Company side	The dashboard starter, TanStack Form	A company posts a job in under 2 minutes and shortlists an applicant.
R7	Challenges and submissions	CTFd as a reference, GitHub App	A candidate submits twice and the company sees both versions.
R8	Admin, security, deploy	Playwright, Vercel	The 12-line end-to-end test passes and RLS is audited.

5. How to run the recovery

1. Copy the kit's files (AGENTS.md, PROMPTS.md, .env.example and the docs folder) into the existing project.
2. Open the project in Antigravity. Start a new conversation, use /plan, and paste **R0** from PROMPTS.md. Read docs/AUDIT.md when it finishes.
3. Paste the **hands-off goal** from PROMPTS.md. It works through R1 to R8, one plan at a time.
4. Fill .env.local yourself when the agent asks for a key. Do not paste keys into the chat.
5. At the end, run the **Final check** prompt and read the pass or fail list.

If the agent hits a usage limit mid-run, docs/PROGRESS.md tells the next conversation exactly where to resume.

Sources

- Reqcore: AI resume parsing explained
- Equip: how parsers extract skills
- Docling
- Resume-Matcher
- OpenResume